Assignment 3: подготовка фестиваля

Представьте, что ваша команда готовит фестиваль в университете. Нужно соединить вход, сцену, зону еды и мастерские кабелями, определить порядок подготовки и организовать доставку. Для этого есть три вида данных: кабель соединяет две площадки в обе стороны; зависимость подготовки направлена от одной задачи к другой; маршрут доставки тоже имеет направление, а ещё стоимость.

Вам нужно завершить небольшую программу, которая отвечает на пять вопросов об этих графах. Каркас программы уже есть в шаблоне GitHub. Ваша работа — реализовать алгоритмы вместо TODO, проверить их и объяснить решение. Выберите **один** язык: Java, Python, C++ или C#. Сайт, интерфейс, база данных и отдельный симулятор фестиваля не нужны.

Фестиваль — это контекст задач. В программе площадки и подготовительные задачи обозначаются номерами вершин: 0, 1, 2 и так далее. Ребро — связь между двумя вершинами. У каждой части свой граф: кабельная сеть и план подготовки не являются одним и тем же входом.

План работы

1. Создайте свою копию шаблона GitHub и выберите язык. Откройте файл Algorithms из таблицы ниже. Ввод и вывод уже реализованы.

2. Завершите DSU: отслеживайте связанные группы при добавлении кабелей.

3. Реализуйте Краскала и Прима: выберите кабельную сеть минимальной стоимости.

4. Реализуйте Косарайю: найдите группы подготовительных задач с циклическими зависимостями.

5. Реализуйте топологическую сортировку через DFS: определите, можно ли составить план подготовки, и верните порядок.

6. Реализуйте кратчайшие пути в DAG: найдите самые дешёвые маршруты доставки и восстановите маршрут.

7. Запустите готовые тесты в GitHub, добавьте собственные примеры и заполните короткий анализ в ANALYSIS.md. Затем сдайте в Moodle ссылку на конкретный коммит и ссылку на успешный запуск его тестов.

Где писать код

Язык	Файл с алгоритмами
Java	Java/source/Algorithms.java
Python	Python/source/algorithms.py
C++	Cpp/source/algorithms.hpp
C#	CSharp/source/Algorithms.cs

Заполните шесть методов dsu, kruskal, prim, scc, topo и dag в файле выбранного языка. Можно добавить вспомогательные функции, классы и исходники непосредственно в ту же папку source. Сохраните работоспособность готовой основной программы и

JSON-ввода/вывода. Реализуется только выбранный язык; остальные папки — альтернативы.

Часть 1. Какие площадки уже соединены?

Между двумя площадками добавляют кабель. Если они были в разных группах, группы объединяются. Если уже были соединены, ничего не меняется. После каждого кабеля верните число связанных групп и размер самой большой.

Напишите dsu на основе системы непересекающихся множеств. Самостоятельно реализуйте find и union со **сжатием путей** и **объединением по размеру или рангу**. Изначально каждая вершина — отдельная группа.

Пример: есть четыре площадки с номерами 0–3. Кабели добавляются по очереди:

Новый кабель	Осталось групп	Самая большая группа
0–1	3	2
1–2	2	3
0–2	2	3
3–3	2	3

Третий кабель ничего не объединяет: 0 и 2 уже связаны через 1. Кабель от 3 к самой себе тоже не меняет ответ. Результат states: [[3,2],[2,3],[2,3],[2,3]].

Поля входа: n — число вершин; operations — список пар вершин. Выход: states, то есть [число групп, размер наибольшей] после каждой операции. Повторные пары и объединение с собой допустимы. При n=0 операций нет, результат пустой; размер наибольшей группы равен 0.

Требуемая сложность: O((n+q) alpha(n)) амортизированного времени для q операций и O(n+q) памяти с учётом результата.

Часть 2. Какие кабели нужно купить?

Теперь у каждого возможного кабеля есть стоимость. Выберите кабели так, чтобы соединить все площадки, которые можно соединить, не образовать циклов и получить минимальную суммарную стоимость. Если граф несвязный, постройте отдельное дерево в каждой связной группе. Вместе эти деревья образуют **минимальный остовный лес**. Изолированная вершина тоже считается компонентой.

Решите задачу двумя отдельными реализациями:

1. kruskal: отсортируйте рёбра по весу и используйте DSU, чтобы не добавлять цикл.

2. prim: используйте списки смежности и очередь с приоритетом для расширения одной компоненты; при необходимости начинайте заново с непосещённой вершины.

Верните cost, edge_ids и components. ID ребра — его позиция во входном списке edges, начиная с 0. Выбранные ID не должны повторяться. Петли не включаются. Параллельные рёбра и отрицательные стоимости допустимы. Если минимальных лесов несколько, принимается любой корректный оптимум.

Пример: n=4, рёбра [0,1,4], [1,2,2], [0,2,3]. Выберите ID 1 и 2: сумма равна 5. Вершина 3 изолирована, поэтому components=2. Оба алгоритма должны получить стоимость 5; при равных оптимумах наборы ID могут различаться.

Вход: n и edges, где ребро записано как [u,v,weight]. Выход: {cost,edge_ids,components}. При n=0 верните cost=0, edge_ids=[] и components=0.

Сравните Краскала и Прима на **одних и тех же графах трёх размеров**. В ANALYSIS.md укажите размеры, способ получения графов, команды запуска или ссылки Actions, не менее трёх повторных измерений и медианное время. Объясните наблюдения. Это сравнение алгоритмов, а не соревнование студентов по скорости.

Требуемая сложность: Краскал O(n+m log(m+1)); Прим O(n+m log(m+1)) с ленивой кучей или O((n+m) log(n+1)) с индексированной. Память: O(n+m).

Часть 3. Какие задачи зависят друг от друга?

Ребро u→v означает, что u нужно завершить до v. Иногда зависимости замыкаются: например, 0 требует 1, а 1 требует 0. Найдите **компоненты сильной связности**: максимальные группы, внутри которых из любой вершины можно попасть в любую другую по направленным рёбрам.

Напишите scc по алгоритму **Косарайю**, с двумя проходами DFS и транспонированным графом. Присвойте каждой вершине метку компоненты. Метки могут быть любыми неотрицательными целыми числами, но одинаковая метка должна означать ровно одну SCC.

Также верните конденсацию: замените каждую SCC одной вершиной и добавьте по одной направленной связи между разными компонентами. Уберите внутренние связи и повторы пар компонент.

Пример: рёбра 0→1, 1→0, 1→2, 0→2, 2→3. Допустимые метки: [5,5,7,9]; конденсация: [[5,7],[7,9]]. Вершины 0 и 1 входят в одну SCC; две связи из этой компоненты к вершине 2 превращаются в одну.

Вход: n и edges вида [u,v]. Выход: component с одной меткой на вершину и condensation с парами меток. Изолированная вершина образует свою компоненту. Петли и повторные рёбра допустимы. При n=0 оба массива пусты. Конденсация является DAG, но найденная SCC не устраняет исходный конфликт расписания внутри неё.

Требуемая сложность: O(n+m) ожидаемого времени при устранении повторов хеш-множеством; O(n+m) памяти. DFS должен работать и на глубоких графах без переполнения стека рекурсии.

Часть 4. В каком порядке готовить фестиваль?

По ориентированному графу зависимостей верните последовательность, в которой каждая задача встречается один раз, а её предварительные задачи стоят раньше. Это **топологический порядок**. При любом направленном цикле такой порядок невозможен.

Напишите topo через DFS с тремя состояниями: не посещена, активна, завершена. Ребро к активной вершине обнаруживает цикл. Для ациклического графа переверните порядок завершения DFS. Не заменяйте этот метод алгоритмом Кана.

Пример: зависимости 0→2, 1→2, 2→3 допускают [1,0,2,3] и [0,1,2,3]. Обе вершины 0 и 1 должны стоять перед 2, а 2 — перед 3.

Вход: n и edges [u,v]. Выход: acyclic и order. Если цикл есть в любой части графа, верните acyclic=false и order=[]. Иначе верните acyclic=true и любой допустимый порядок всех вершин. Для пустого графа: true и []. Учитывайте изолированные вершины, петли, повторы рёбер и глубокие графы.

Требуемая сложность: O(n+m) времени и памяти, включая стек DFS.

Часть 5. Как дешевле доставить материалы?

У направленного графа доставки есть стоимость каждого перехода. Из заданной вершины source найдите самые дешёвые маршруты ко всем вершинам. В этой части нужен **DAG**: ориентированный граф без циклов. Отрицательная стоимость допустима; например, переход может означать экономию.

Напишите dag. Сначала проверьте ацикличность **всего графа**. Цикл даже в недостижимой области делает вход неподходящим: верните acyclic=false, distance=[] и parent_edge=[]. Иначе обрабатывайте вершины в топологическом порядке и релаксируйте исходящие рёбра. Не релаксируйте из недостижимой вершины.

Верните distance с минимальной стоимостью для каждой вершины и parent_edge с ID последнего ребра выбранного кратчайшего пути. У источника distance=0 и parent_edge=-1. У недостижимой вершины distance=null и parent_edge=-1. Остальные родительские рёбра должны восстанавливать корректные кратчайшие пути из источника. При равных стоимостях принимается любой кратчайший путь.

Пример: source=0, n=5, рёбра [0,1,3], [0,2,8], [1,2,-7], [2,3,4]. Расстояния: [0,3,-4,0,null]; допустимый parent_edge: [-1,0,2,3,-1]. Маршрут к 3: 0→1→2→3, стоимость 3−7+4=0. Вершина 4 недостижима.

Добавьте вспомогательную функцию, которая восстанавливает последовательность вершин по parent_edge. В собственных тестах или анализе покажите её работу для достижимой вершины, самого источника и недостижимой вершины. Остовный лес из части 2 не решает задачу кратчайшего пути.

Вход: n, edges [u,v,weight] и source. Выход: {acyclic,distance,parent_edge}. Параллельные рёбра допустимы; петля является циклом. Стоимости и расстояния должны поддерживать знаковые 64-битные целые числа.

Требуемая сложность: O(n+m) времени и памяти; восстановление пути — O(длины пути).

Как проверить работу в GitHub

В репозитории уже есть примеры и готовые тесты. Их можно запустить через GitHub Actions без установки Python для инструментов проверки.

1. На странице шаблона выберите Use this template → Create a new repository. Работайте в своей копии. Она может быть публичной или приватной; к приватной преподавателю нужен доступ.

2. В submission.json выберите language: Java, Python, Cpp или CSharp. Для C++ пишите Cpp, для C# — CSharp. Заполните файл Algorithms своего языка. Можно работать локально в IDE, затем делать commit и push.

3. В своём репозитории откройте Actions → Public checks. После push проверка запускается автоматически. Если GitHub предложит, включите Actions. Откройте последний запуск вашего коммита и изучите шаги: Compile and launch проверяет запуск программы, Algorithm examples — алгоритмы, включая границы и повторные запросы. Пока есть TODO, красные ошибки ожидаемы. Исправьте первую подходящую ошибку и сделайте новый push.

4. Добавьте собственные примеры в tests/student/: вход, ожидаемый и фактический результат. Заполните ANALYSIS.md: по 3–5 предложений на алгоритм, сравнение Краскала и Прима, пример восстановления маршрута и раскрытие помощи по силлабусу.

5. Перед сдачей проверьте, что Public checks зелёный именно для сдаваемого коммита. Это готовые тесты для подготовки. После дедлайна преподаватель выполнит отдельную, более глубокую проверку этого коммита и посмотрит требуемые методы. Успешные публичные тесты не гарантируют правильность всех возможных случаев.

Для локального запуска TESTING.md объясняет необязательный launcher. Python-launcher — инструмент проверки всех четырёх языков, а не требование писать решение на Python. В CONTRACTS.md есть точные JSON-поля и полные границы входа, если понадобится технический справочник. На каждый запрос нужен один JSON-ответ, состояние между запросами сбрасывается; отладочные сообщения не должны попадать в JSON-вывод программы.

Что сдавать

В Online text Moodle вставьте **две ссылки**:

1. На конкретный коммит с готовым кодом, анализом и тестами: https://github.com/YOUR_ACCOUNT/YOUR_REPOSITORY/commit/FULL_40_CHARACTER_SHA . Откройте коммит в GitHub и скопируйте его URL; полный SHA фиксирует проверяемую версию.

2. На успешный Public checks этого же коммита: https://github.com/YOUR_ACCOUNT/YOUR_REPOSITORY/actions/runs/RUN_ID . Откройте зелёный запуск и скопируйте URL. Его коммит должен совпадать с первой ссылкой.

Сохраните, нажмите Submit assignment и убедитесь, что появился Submitted for grading. Весь проект загружать не нужно, ID студента вводить не нужно. Одной изменяемой ссылки на main недостаточно. Коммит определяет работу, ссылка Actions подтверждает прохождение публичных тестов, Moodle фиксирует время сдачи. Если исправляете работу до закрытия приёма, сдайте новый коммит и ссылку на его тесты через доступные элементы; если работа заблокирована, обратитесь к преподавателю. Поздние изменения репозитория не заменяют сданный коммит.